Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования

«Пермский национальный исследовательский политехнический университет»

Электротехнический факультет

Выпускающая кафедра: информационные технологии и автоматизированные системы (ИТАС)

Направление подготовки: 09.03.01 Информатика и вычислительная техника

ОТЧЕТ

Лабораторная работа № 8

По дисциплине «Теория алгоритмов и структуры данных»

Вариант № 25

Выполнил: студент группы ИВТ-25-2б
Соловьев Тимофей Николаевич

Принял: Доц. Полякова О. А.

Пермь, 2026

1. Постановка задачи

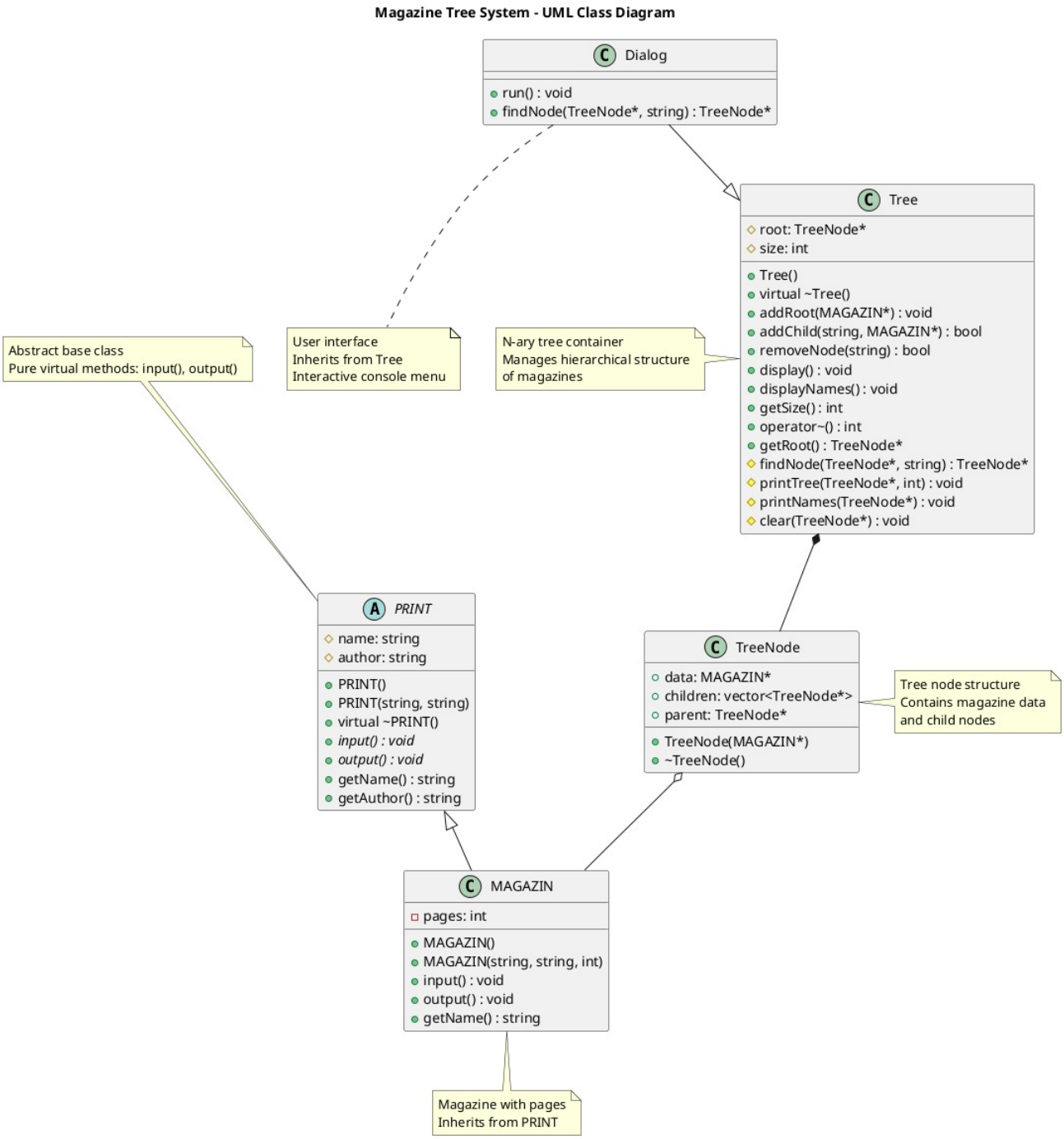
```
Постановка задачи
1. Определить иерархию пользовательских классов (см. лабораторную работу No5).
Во главе иерархии должен стоять абстрактный класс с чисто виртуальными
методами для ввода и вывода информации об атрибутах объектов.
2. Реализовать конструкторы, деструктор, операцию присваивания, селекторы и
модификаторы.
3. Определить класс-группу на основе структуры, указанной в варианте.
4. Для группы реализовать конструкторы, деструктор, методы для добавления и
удаления элементов в группу, метод для просмотра группы, перегрузить операцию
для получения информации о размере группы.
5. Определить класс Диалог – наследника группы, в котором реализовать методы для
обработки событий.
6. Добавить методы для обработки событий группой и объектами пользовательских
классов.
7. Написать тестирующую программу.
8. Нарисовать диаграмму классов и диаграмму объектов.
Базовый класс:
ПЕЧАТНОЕ_ИЗДАНИЕ(PRINT)
Название– string
Автор – string
Производный класс
ЖУРНАЛ (MAGAZIN)
Количество страниц - int
Группа – Дерево (Tree).
Команды:
Создать группу (формат команды: m количество элементов группы).
Добавить элемент в группу (формат команды: +)
Удалить элемент из группы (формат команды -)
Вывести информацию об элементах группы (формат команды: s)
Вывести информацию о названиях всех элементов группы (формат команды : z)
Конец работы (формат команды: q)
```

2. Анализ

```
Требуется разработать иерархию классов для представления печатных изданий. Абстрактный базовый класс
PRINT должен содержать защищенные поля: название издания в виде строки и автора в виде строки.
Конструктор по умолчанию инициализирует обе строки пустыми значениями. Конструктор с параметрами
принимает название и автора. Деструктор должен быть виртуальным для обеспечения корректного разрушения
объектов производных классов. В классе определяются два чистых виртуальных метода: input для ввода данных
и output для вывода информации об объекте. Также определяются виртуальные методы getName и getAuthor для
доступа к полям.

Производный класс MAGAZIN наследует от PRINT и добавляет закрытое поле pages для хранения количества
страниц. Конструктор без параметров вызывает конструктор базового класса и инициализирует страницы нулем.
Конструктор с параметрами принимает название, автора и количество страниц. Метод input переопределяется и
последовательно запрашивает ввод названия, автора и количества страниц. Метод output переопределяется и
выводит информацию о журнале в квадратных скобках. Метод getName возвращает название.

Для организации группы объектов на основе структуры дерева определяется структура TreeNode. Каждый узел
содержит указатель на объект MAGAZIN, вектор указателей на дочерние узлы children и указатель на
родительский узел parent. Конструктор инициализирует данные и обнуляет родителя. Деструктор удаляет
данные журнала и рекурсивно всех потомков.

Класс Tree является основным классом группы. Он содержит защищенные поля: root указатель на корневой узел
и size целочисленное количество элементов. Конструктор инициализирует корень нулем и размер нулем.
Деструктор вызывает метод clear для рекурсивного удаления всех узлов. Метод addRoot создает корневой узел
из переданного журнала, если корень еще не существует. Метод addChild находит узел с заданным именем и
добавляет к нему новый дочерний узел. Метод removeNode удаляет узел по имени с учетом того, что корень
удаляется отдельно. Метод findNode выполняет рекурсивный поиск узла по названию. Методы printTree и
printNames рекурсивно выводят информацию о структуре и о названиях журналов. Метод display вызывает
printTree. Метод displayNames вызывает printNames. Метод clear рекурсивно удаляет все узлы. Оператор
тильда возвращает размер дерева.

Класс Dialog наследует от Tree и реализует интерактивное меню для работы с деревом. Метод run содержит
цикл обработки команд. Команда m создает корневой журнал, если корень отсутствует. Команда плюс добавляет
дочерний журнал к указанному родительскому узлу. Команда минус удаляет журнал по указанному имени.
Команда s выводит полную структуру дерева с отступами. Команда z выводит только названия всех журналов.
Команда q завершает программу.

В основной программе создается объект класса Dialog и вызывается его метод run. Пользователь
последовательно вводит команды для создания дерева, добавления и удаления узлов, просмотра структуры.
Программа поддерживает вывод текущего размера группы после каждой операции. Все методы ввода и вывода
```

```
объектов журнала используют переопределенные виртуальные функции, что обеспечивает полиморфное поведение.
При необходимости в будущем могут быть добавлены другие производные классы от PRINT, которые будут
корректно обрабатываться существующей структурой дерева.
```

3. UML-диаграмма



4. Исходный код

Файл реализации (.cpp)

```
#include
#include
#include
#include

using namespace std;

class PRINT {
protected:
    string name;
    string author;
public:
    PRINT() : name(""), author("") {}
```

```
    PRINT(string n, string a) : name(n), author(a) {}
    virtual ~PRINT() {}
    virtual void input() = 0;
    virtual void output() const = 0;
    virtual string getName() const { return name; }
    virtual string getAuthor() const { return author; }
};

class MAGAZIN : public PRINT {
private:
    int pages;
public:
    MAGAZIN() : PRINT(), pages(0) {}
    MAGAZIN(string n, string a, int p) : PRINT(n, a), pages(p) {}
    void input() override {
        cout << "Enter name: "; cin >> name;
        cout << "Enter author: "; cin >> author;
        cout << "Enter pages: "; cin >> pages;
    }
    void output() const override {
        cout << "[" << name << ", " << author << ", " << pages << "p]";
    }
    string getName() const { return name; }
};

struct TreeNode {
    MAGAZIN* data;
    vector children;
    TreeNode* parent;

    TreeNode(MAGAZIN* m) : data(m), parent(nullptr) {}
    ~TreeNode() { delete data; for(auto child : children) delete child; }
};

class Tree {
protected:
    TreeNode* root;
    int size;

    TreeNode* findNode(TreeNode* node, const string& name) {
        if(!node) return nullptr;
        if(node->data->getName() == name) return node;
        for(auto child : node->children) {
            TreeNode* found = findNode(child, name);
            if(found) return found;
        }
        return nullptr;
    }

    void printTree(TreeNode* node, int level) {
        if(!node) return;
        for(int i = 0; i < level; i++) cout << "  ";
        node->data->output();
        cout << endl;
        for(auto child : node->children) {
            printTree(child, level + 1);
        }
    }

    void printNames(TreeNode* node) {
        if(!node) return;
        for(int i = 0; i < 2; i++) cout << " ";
        cout << node->data->getName() << endl;
        for(auto child : node->children) {
            printNames(child);
        }
    }

    void clear(TreeNode* node) {
        if(node) {
            for(auto child : node->children) clear(child);
            delete node;
        }
    }

public:
    Tree() : root(nullptr), size(0) {}
    ~Tree() { clear(root); }
```

```
    void addRoot(MAGAZIN* m) {
        if(!root) {
            root = new TreeNode(m);
            size++;
        }
    }

    bool addChild(const string& parentName, MAGAZIN* m) {
        TreeNode* parent = findNode(root, parentName);
        if(parent) {
            TreeNode* child = new TreeNode(m);
            child->parent = parent;
            parent->children.push_back(child);
            size++;
            return true;
        }
        return false;
    }

    bool removeNode(const string& name) {
        if(!root) return false;
        if(root->data->getName() == name) {
            clear(root);
            root = nullptr;
            size = 0;
            return true;
        }
        TreeNode* node = findNode(root, name);
        if(node && node->parent) {
            auto& siblings = node->parent->children;
            siblings.erase(std::remove(siblings.begin(), siblings.end(), node), siblings.end());
            delete node;
            size--;
            return true;
        }
        return false;
    }

    void display() {
        if(!root) cout << "Empty tree" << endl;
        else {
            cout << "\n=== Tree Structure ===" << endl;
            printTree(root, 0);
        }
    }

    void displayNames() {
        if(!root) cout << "Empty tree" << endl;
        else {
            cout << "\n=== Magazine Names ===" << endl;
            printNames(root);
        }
    }

    int getSize() const { return size; }
    int operator~() const { return size; }
    TreeNode* getRoot() { return root; }
};

class Dialog : public Tree {
public:
    void run() {
        char cmd;
        cout << "=== Magazine Tree Manager ===" << endl;
        cout << "First, create root node!" << endl;

        while (true) {
            cout << "\nCommands:" << endl;
            cout << "  m   - create root magazine" << endl;
            cout << "  +   - add child magazine (asks for parent name)" << endl;
            cout << "  -   - remove magazine" << endl;
            cout << "  s   - show tree structure" << endl;
            cout << "  z   - show magazine names" << endl;
            cout << "  q   - quit" << endl;
            cout << "Enter command: ";
            cin >> cmd;

            if (cmd == 'm') {
                if(root) {
```

```
                    cout << "Root already exists! Use + to add children." << endl;
                } else {
                    MAGAZIN* m = new MAGAZIN();
                    cout << "--- Create Root Magazine ---" << endl;
                    m->input();
                    addRoot(m);
                    cout << "\nRoot created. Size: " << ~(*this) << endl;
                }
            }
            else if (cmd == '+') {
                if(!root) {
                    cout << "Create root first using 'm' command!" << endl;
                    continue;
                }
                string parentName;
                cout << "Enter parent magazine name: "; cin >> parentName;
                if(!findNode(root, parentName)) {
                    cout << "Parent not found!" << endl;
                    continue;
                }
                MAGAZIN* m = new MAGAZIN();
                cout << "--- Create Child Magazine ---" << endl;
                m->input();
                if(addChild(parentName, m)) {
                    cout << "\nAdded child to '" << parentName << "'. Size: " << ~(*this) << endl;
                }
            }
            else if (cmd == '-') {
                if(!root) {
                    cout << "Tree is empty!" << endl;
                    continue;
                }
                string name;
                cout << "Enter name to remove: "; cin >> name;
                if(removeNode(name)) cout << "\nRemoved. Size: " << ~(*this) << endl;
                else cout << "\nNot found or cannot remove" << endl;
            }
            else if (cmd == 's') {
                display();
            }
            else if (cmd == 'z') {
                displayNames();
            }
            else if (cmd == 'q') {
                cout << "\nGoodbye!" << endl;
                break;
            }
            else {
                cout << "\nUnknown command!" << endl;
            }
        }
    }

    TreeNode* findNode(TreeNode* node, const string& name) {
        if(!node) return nullptr;
        if(node->data->getName() == name) return node;
        for(auto child : node->children) {
            TreeNode* found = findNode(child, name);
            if(found) return found;
        }
        return nullptr;
    }
};

int main() {
    Dialog dlg;
    dlg.run();
    return 0;
}
```

5. Результаты выполнения

```
=== Magazine Tree Manager ===
First, create root node!

Commands:
```

```
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: m
--- Create Root Magazine ---
Enter name: rootMagazin
Enter author: me
Enter pages: 123

Root created. Size: 1

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: +
Enter parent magazine name: rootMagazin
--- Create Child Magazine ---
Enter name: special
Enter author: goodGuy
Enter pages: 234

Added child to 'rootMagazin'. Size: 2

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: +
Enter parent magazine name: rootMagazin
--- Create Child Magazine ---
Enter name: halloween
Enter author: vampire
Enter pages: 167

Added child to 'rootMagazin'. Size: 3

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: +
Enter parent magazine name: halloween
--- Create Child Magazine ---
Enter name: special
Enter author: vampire
Enter pages: 169

Added child to 'halloween'. Size: 4

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: s

=== Tree Structure ===
[rootMagazin, me, 123p]
  [special, goodGuy, 234p]
  [halloween, vampire, 167p]
    [special, vampire, 169p]

Commands:
  m   - create root magazine
```

```
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: -
Enter name to remove: halloween

Removed. Size: 3

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: s

=== Tree Structure ===
[rootMagazin, me, 123p]
  [special, goodGuy, 234p]

Commands:
  m   - create root magazine
  +   - add child magazine (asks for parent name)
  -   - remove magazine
  s   - show tree structure
  z   - show magazine names
  q   - quit
Enter command: q

Goodbye!
```

6. Ответы на вопросы

```
Класс-группа — это класс, предназначенный для хранения и управления набором однородных или разнородных
объектов. Примеры: список, стек, очередь, массив, множество, словарь.

Пример описания класса List:
class List {
private:
    struct Node {
        int data;
        Node* next;
    };
    Node* head;
    int size;
public:
    List();
    List(int value);
    List(const List& other);
    ~List();
    void display();
};

Конструктор без параметров: List::List() : head(nullptr), size(0) {}
Конструктор с параметром: List::List(int value) : head(nullptr), size(0) { head = new Node{value,
nullptr}; size = 1; }
Конструктор копирования: List::List(const List& other) : head(nullptr), size(0) { Node* current =
other.head; while (current) { add(current->data); current = current->next; } }

Деструктор: List::~List() { while (head) { Node* temp = head; head = head->next; delete temp; } size = 0;
}

Метод для просмотра элементов: void List::display() { Node* current = head; while (current) { cout <<
current->data << " "; current = current->next; } cout << endl; }

Группа дает иерархию типа «часть-целое» (агрегацию или композицию), а не наследование. Это иерархия
контейнеров, где один объект содержит другие.

Абстрактный класс задает общий интерфейс для всех объектов, которые могут храниться в группе. Это
позволяет группе работать с разнородными объектами единообразно через указатели или ссылки на базовый
класс, не зная их конкретных типов.
```

Событие — это факт возникновения некоторого действия. События используются для уведомления одних компонентов программы о действиях других компонентов или пользователя, обеспечивая реактивность и асинхронное взаимодействие.

Характеристики события-сообщения: тип события, источник события, время возникновения, данные (параметры события) и, возможно, адресат.

Пример структуры, описывающей событие: struct Event { int type; int x; int y; int keyCode; };

Полю what присваиваются числовые коды, идентифицирующие тип события. Например: evMouseMove — движение мыши, evMouseDown — нажатие кнопки мыши, evKeyDown — нажатие клавиши, evMessage — пользовательское сообщение.

Полю command присваиваются значения (коды команд), когда what указывает на событие типа команда. Например: cmOpen — открыть файл, cmSave — сохранить, cmExit — выход из программы.

Поля a и message используются для передачи дополнительных параметров команды. message может содержать текст сообщения или дополнительный код подкоманды, а a — числовой параметр.

Необходимы методы: GetEvent() для получения события, DispatchEvent() или HandleEvent() для маршрутизации и обработки события, ClearEvent() для очистки или сброса события, а также метод проверки наличия событий.

Главный цикл обработки событий:

```
while (running) {
    if (GetEvent(&event)) {
        HandleEvent(&event);
        if (event.type == EV_QUIT) running = false;
    } else {
        // idle actions
    }
    ClearEvent(&event);
}
```

ClearEvent() очищает или сбрасывает событие после его обработки. Обычно он обнуляет поля структуры события или удаляет его из очереди, чтобы избежать повторной обработки и освободить ресурсы.

HandleEvent() обрабатывает событие. Он анализирует тип события (what) и, в зависимости от него, вызывает соответствующие обработчики.

GetEvent() извлекает следующее событие из очереди событий и заполняет переданную структуру события. Возвращает true, если событие получено, и false, если событий нет.

Поле EndState используется в классах диалогов или окон для указания того, какой элемент завершил диалог. Содержит его обычно класс TDialog.

Функция Valid() проверяет корректность данных, введенных пользователем, прежде чем закрыть диалог или перейти к следующему шагу. Если проверка не пройдена, Valid() возвращает false и не позволяет закрыть окно.